tb_arith

June 2, 2026

[15]:
```
from __future__ import division
import numpy as np
import matplotlib.pyplot as plt

intp = 10  #tomar uno de cada cien valores, para graficar rapido

# Load data from the .txt file
#ssh -Y cic@148.204.66.123
#ssh -Y alex@148.204.66.53

#source ~/miniconda3/bin/activate
#python3 tb_4x8x4_proc.py
#scp alex@148.204.66.53:'/home/cic/Desktop/EDA/SNN_IPN/sim_results/*.pdf' .
#scp cic@148.204.66.123:'/home/cic/Desktop/EDA/SNN_IPN/sim_results/*.pdf' .

#scp alex@148.204.66.53:/home/alex/.xschem/simulations/data.raw .
#scp alex@148.204.66.53:/home/alex/Desktop/EDA/SNN_IPN/sim_results/
 ↪tb_4x8x4_data.txt .
#scp cic@148.204.66.123:/home/cic/Desktop/EDA/SNN_IPN/sim_results/tb_4x8x4_data.
 ↪txt  .

BSIZE_SP = 512 # Max size of a line of data; we don't want to read the
               # whole file to find a line, in case file does not have
               # expected structure.

MDATA_LIST = [b'title', b'date', b'plotname', b'flags', b'no. variables',
              b'no. points', b'dimensions', b'command', b'option']

def rawread(fname: str):
    """Read ngspice binary raw files. Return tuple of the data, and the
    plot metadata. The dtype of the data contains field names. This is
    not very robust yet, and only supports ngspice.

    >>> darr, mdata = rawread('test.py')
    >>> darr.dtype.names
    >>> plot(np.real(darr['frequency']), np.abs(darr['v(out)']))
    """
```

```
    # Example header of raw file
    # Title: rc band pass example circuit
    # Date: Sun Feb 21 11:29:14  2016
    # Plotname: AC Analysis
    # Flags: complex
    # No. Variables: 3
    # No. Points: 41
    # Variables:
    #         0       frequency       frequency       grid=3
    #         1       v(out)  voltage
    #         2       v(in)   voltage
    # Binary:
    fp = open(fname, 'rb')
    arrs = []
    plots = []
    plot = {}
    while (True):
        try:
            # mdata = fp.readline(BSIZE_SP).split(b':', maxsplit=1)
            mdata = fp.readline().split(b':', maxsplit=1)
        except:
            raise
        if len(mdata) == 2:
            if mdata[0].lower() in MDATA_LIST:
                plot[mdata[0].lower()] = mdata[1].strip()
            if mdata[0].lower() == b'variables':
                nvars = int(plot[b'no. variables'])
                npoints = int(plot[b'no. points'])
                plot['varnames'] = []
                plot['varunits'] = []
                for varn in range(nvars):
                    # varspec = (fp.readline(BSIZE_SP).strip().decode('ascii').
↪split())
                    varspec = (fp.readline().strip().decode('ascii').split())
                    assert(varn == int(varspec[0]))
                    plot['varnames'].append(varspec[1])
                    plot['varunits'].append(varspec[2])
            if mdata[0].lower() == b'binary':
                rowdtype = np.dtype({'names': plot['varnames'],
                                     'formats': [np.complex_ if b'complex'
                                                 in plot[b'flags']
                                                 else np.float64]*nvars})
                # We should have all the metadata by now
                arrs.append(np.fromfile(fp, dtype=rowdtype, count=npoints))
                plots.append(plot)
                plot = {} # reset the plot dict
                fp.readline() # Read to the end of line
```

```
        else:
            break
    return (arrs[0], plots)

data, dicc = rawread("data_arith.raw")
```

[16]:
```
data
```

[16]:
```
array([(6.25000000e-11, 1.6393422 , 1.63906843,  9.12160658e-01,
9.12160588e-01,  9.12160729e-01,  9.12160519e-01, 6.59757898e-01,  0.66103983,
4.69078681e-06,  4.67276296e-06,  4.69072077e-06,  4.67269710e-06,
4.69085287e-06,  4.67282882e-06,  4.69065473e-06,  4.67263126e-06,
5.48102394e-05, 5.48102063e-05, 5.48102723e-05, 5.48101731e-05, 5.28609306e-05,
5.28320931e-05, 0.8       , 0.75000003, 0.93727545, 0.9270664 , 0.72999998,
0.63500002,  0.07051042,  0.02337337,  0.07040666,  0.02345   ,  0.07050973,
0.02337335,  0.07040598,  0.02344998,  0.0705111 ,  0.0233734 ,  0.07040735,
0.02345003,  0.07050904,  0.02337332,  0.07040529,  0.02344995, 0.01838506,
0.01838505, 0.01838506, 0.01838505, 0.01823516, 0.01823558),
       (6.42577674e-11, 1.63893279, 1.63867928,  9.25497302e-01,
9.25497184e-01,  9.25497421e-01,  9.25497067e-01, 6.69792304e-01,  0.67110104,
4.83486679e-06,  4.81633695e-06,  4.83479999e-06,  4.81627035e-06,
4.83493360e-06,  4.81640356e-06,  4.83473321e-06,  4.81620377e-06,
3.53262010e-06, 3.53282143e-06, 3.53241891e-06, 3.53302290e-06, 6.65165325e-06,
6.61501553e-06, 0.8       , 0.75000003, 0.93738359, 0.92718653, 0.72999998,
0.63500002,  0.07224218,  0.02361282,  0.07213416,  0.02369039,  0.07224148,
0.02361279,  0.07213346,  0.02369036,  0.07224288,  0.02361286,  0.07213486,
0.02369043,  0.07224078,  0.02361276,  0.07213276,  0.02369033, 0.01841734,
0.01841734, 0.01841734, 0.01841734, 0.01827192, 0.01827234),
       (6.77733022e-11, 1.63674755, 1.63652149,  9.51618131e-01,
9.51617916e-01,  9.51618347e-01,  9.51617702e-01, 6.89620844e-01,  0.69098314,
5.20190837e-06,  5.18162199e-06,  5.20183889e-06,  5.18155272e-06,
5.20197787e-06,  5.18169127e-06,  5.20176942e-06,  5.18148347e-06,
1.05878754e-06, 1.05897852e-06, 1.05859674e-06, 1.05916968e-06, 3.94385250e-06,
3.90657616e-06, 0.8       , 0.75000003, 0.93709785, 0.9269101 , 0.72999998,
0.63500002,  0.07653628,  0.02417228,  0.07641116,  0.02425037,  0.07653554,
0.02417223,  0.07641042,  0.02425033,  0.07653702,  0.02417232,  0.0764119 ,
0.02425042,  0.0765348 ,  0.02417219,  0.07640968,  0.02425028, 0.01842869,
0.01842869, 0.01842869, 0.01842869, 0.01829196, 0.01829239),
       …,
       (1.99999082e-02, 0.45795022, 0.86704793,  1.01785459e-04,
2.96486156e-05,  4.07991030e-05,  3.87000675e-05, 4.29741137e-07, -0.01078392,
8.19955885e-09,  5.81521549e-07,  1.53289737e-11,  4.07563349e-07,
-2.16206363e-11,  5.41037379e-07,  3.67349829e-11,  5.21690406e-07,
9.91819971e-08, 1.08041146e-07, 1.91117232e-07, 1.36899909e-08, 2.05193536e-06,
5.16314290e-07, 0.79999998, 1.20645388, 1.21247604, 0.98126247, 0.45612766,
0.86322695, -0.19668736, -0.19679602, -0.10597912, -0.11269097, -0.0712256 ,
```

```
-0.07122661, -0.13587445, -0.14333032, -0.15065399, -0.15065348, -0.13927572,
-0.14569648, -0.02294541, -0.02294596, -0.13765891, -0.14435677, 0.65296308,
1.0568588 , 1.29851724, 1.43585173, 1.78364526, 1.78367072),
       (1.99999541e-02, 0.45786782, 0.86715944, -1.00863177e-04,
-2.83590425e-05, -3.90551300e-05, -3.79197238e-05, 5.84780622e-08,  0.01085472,
-8.14013942e-09, -5.77764342e-07, -1.60841168e-11, -4.03451521e-07,
2.08123748e-11, -5.37403441e-07, -3.76374578e-11, -5.17928466e-07,
8.89891626e-08, 1.01284622e-07, 1.84108481e-07, 6.78387899e-09, 2.05159130e-06,
3.57348342e-06, 0.8       , 1.20646325, 1.20725629, 0.98901141, 0.45612205,
0.86323163, -0.19556834, -0.19546047, -0.11127634, -0.10460755, -0.07121988,
-0.07121882, -0.14176272, -0.13438207, -0.15040897, -0.15040946, -0.1441261 ,
-0.13774833, -0.02295428, -0.02295371, -0.14279513, -0.13614555, 0.65312137,
1.06137112, 1.30642723, 1.435825  , 1.78364639, 1.78373829),
       (2.00000000e-02, 0.45778541, 0.86726434,  1.00978235e-04,
2.89136736e-05,  4.01185523e-05,  3.80027880e-05, 4.39756817e-07, -0.01074962,
8.08835326e-09,  5.74037018e-07,  1.68338762e-11,  3.99380643e-07,
-1.99825788e-11,  5.33795477e-07,  3.85441499e-11,  5.14194809e-07,
9.91129389e-08, 1.08090212e-07, 1.91175590e-07, 1.37370728e-08, 2.05165931e-06,
5.10997107e-07, 0.80000002, 1.20647263, 1.20203298, 0.99626566, 0.45611644,
0.86323631, -0.19425073, -0.19435791, -0.10588229, -0.11250837, -0.07119344,
-0.07119455, -0.13559494, -0.14290113, -0.15014543, -0.15014495, -0.1389287 ,
-0.14526379, -0.02293958, -0.02294015, -0.13733617, -0.14393784, 0.65356953,
1.06587902, 1.31432756, 1.43579955, 1.78364754, 1.78366033)],
      shape=(502164,), dtype=[('time', '<f8'), ('v(hx1)', '<f8'), ('v(hx2)',
'<f8'), ('v(j1)', '<f8'), ('v(j2)', '<f8'), ('v(j3)', '<f8'), ('v(j4)', '<f8'),
('v(k1)', '<f8'), ('v(k2)', '<f8'), ('i(v.x11.xstdp1.vmr)', '<f8'),
('i(v.x11.xstdp2.vmr)', '<f8'), ('i(v.x11.xstdp3.vmr)', '<f8'),
('i(v.x11.xstdp4.vmr)', '<f8'), ('i(v.x11.xstdp5.vmr)', '<f8'),
('i(v.x11.xstdp6.vmr)', '<f8'), ('i(v.x11.xstdp7.vmr)', '<f8'),
('i(v.x11.xstdp8.vmr)', '<f8'), ('i(v.xx1.vext)', '<f8'), ('i(v.xx2.vext)',
'<f8'), ('i(v.xx3.vext)', '<f8'), ('i(v.xx4.vext)', '<f8'), ('i(v.xx5.vext)',
'<f8'), ('i(v.xx6.vext)', '<f8'), ('v(vin1)', '<f8'), ('v(vin2)', '<f8'),
('v(vr1)', '<f8'), ('v(vr2)', '<f8'), ('v(x1)', '<f8'), ('v(x2)', '<f8'),
('v(x11.xstdp1.be)', '<f8'), ('v(x11.xstdp1.te)', '<f8'), ('v(x11.xstdp2.be)',
'<f8'), ('v(x11.xstdp2.te)', '<f8'), ('v(x11.xstdp3.be)', '<f8'),
('v(x11.xstdp3.te)', '<f8'), ('v(x11.xstdp4.be)', '<f8'), ('v(x11.xstdp4.te)',
'<f8'), ('v(x11.xstdp5.be)', '<f8'), ('v(x11.xstdp5.te)', '<f8'),
('v(x11.xstdp6.be)', '<f8'), ('v(x11.xstdp6.te)', '<f8'), ('v(x11.xstdp7.be)',
'<f8'), ('v(x11.xstdp7.te)', '<f8'), ('v(x11.xstdp8.be)', '<f8'),
('v(x11.xstdp8.te)', '<f8'), ('v(xx1.vm)', '<f8'), ('v(xx2.vm)', '<f8'),
('v(xx3.vm)', '<f8'), ('v(xx4.vm)', '<f8'), ('v(xx5.vm)', '<f8'), ('v(xx6.vm)',
'<f8')])
```

[17]:
```
dicc
```

[17]:
```
[{b'title': b'** sch_path:
/foss/designs/snn_ipn/sim_results/arith/tb_4x2_arith.sch',
```

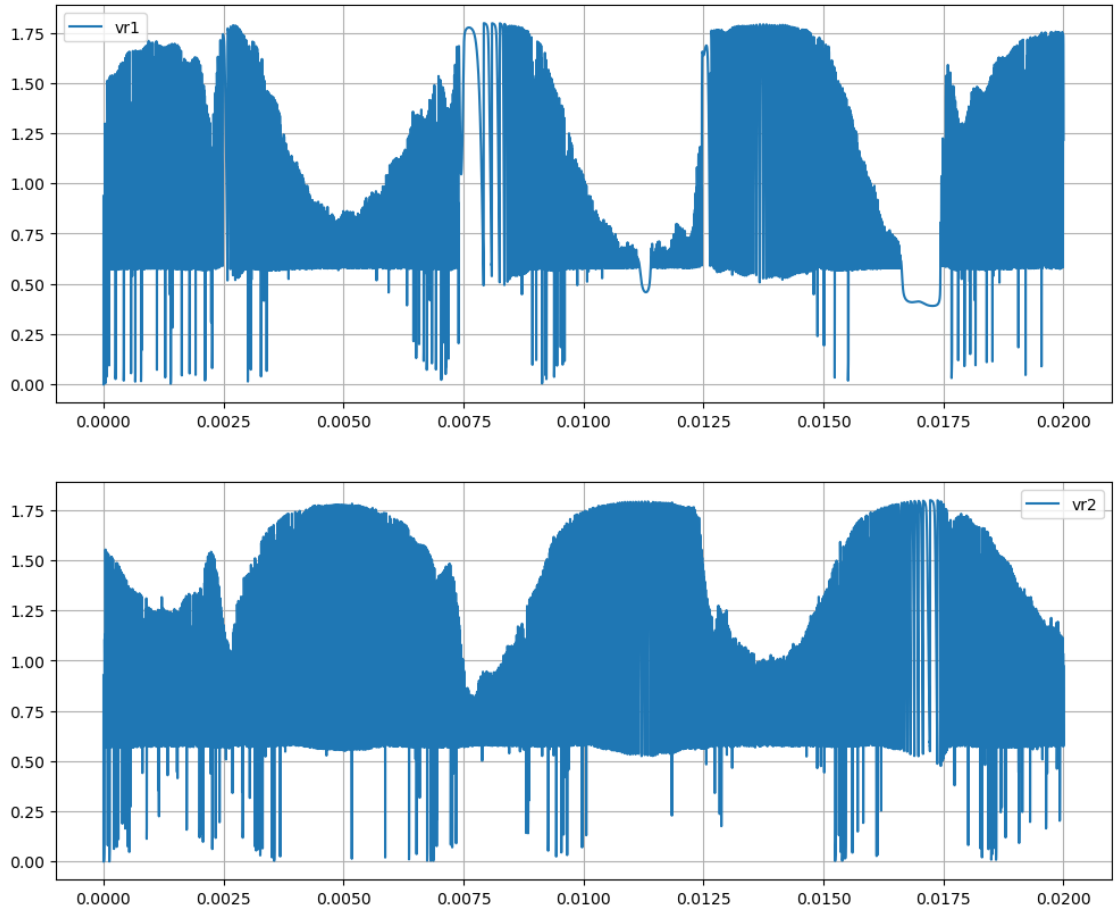
```
b'date': b'Tue Jun  2 18:10:01  2026',
b'command': b'ngspice-46, Build Thu Apr 16 19:53:00 UTC 2026',
b'plotname': b'Transient Analysis',
b'flags': b'real',
b'no. variables': b'51',
b'no. points': b'502164',
'varnames': ['time',
 'v(hx1)',
 'v(hx2)',
 'v(j1)',
 'v(j2)',
 'v(j3)',
 'v(j4)',
 'v(k1)',
 'v(k2)',
 'i(v.x11.xstdp1.vmr)',
 'i(v.x11.xstdp2.vmr)',
 'i(v.x11.xstdp3.vmr)',
 'i(v.x11.xstdp4.vmr)',
 'i(v.x11.xstdp5.vmr)',
 'i(v.x11.xstdp6.vmr)',
 'i(v.x11.xstdp7.vmr)',
 'i(v.x11.xstdp8.vmr)',
 'i(v.xx1.vext)',
 'i(v.xx2.vext)',
 'i(v.xx3.vext)',
 'i(v.xx4.vext)',
 'i(v.xx5.vext)',
 'i(v.xx6.vext)',
 'v(vin1)',
 'v(vin2)',
 'v(vr1)',
 'v(vr2)',
 'v(x1)',
 'v(x2)',
 'v(x11.xstdp1.be)',
 'v(x11.xstdp1.te)',
 'v(x11.xstdp2.be)',
 'v(x11.xstdp2.te)',
 'v(x11.xstdp3.be)',
 'v(x11.xstdp3.te)',
 'v(x11.xstdp4.be)',
 'v(x11.xstdp4.te)',
 'v(x11.xstdp5.be)',
 'v(x11.xstdp5.te)',
 'v(x11.xstdp6.be)',
 'v(x11.xstdp6.te)',
```

```
 'v(x11.xstdp7.be)',
 'v(x11.xstdp7.te)',
 'v(x11.xstdp8.be)',
 'v(x11.xstdp8.te)',
 'v(xx1.vm)',
 'v(xx2.vm)',
 'v(xx3.vm)',
 'v(xx4.vm)',
 'v(xx5.vm)',
 'v(xx6.vm)'],
'varunits': ['time',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'current',
 'current',
 'current',
 'current',
 'current',
 'current',
 'current',
 'current',
 'current',
 'current',
 'current',
 'current',
 'current',
 'current',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
 'voltage',
```

```
        'voltage',
        'voltage',
        'voltage',
        'voltage',
        'voltage',
        'voltage',
        'voltage',
        'voltage',
        'voltage',
        'voltage',
        'voltage',
        'voltage',
        'voltage',
        'voltage']}]
```

[39]:
```
fig, (ax1, ax2) = plt.subplots(2, figsize=(12,10))
ax1.plot(data["time"][::intp], data["v(vr1)"][::intp], label = "vr1")
ax2.plot(data["time"][::intp], data["v(vr2)"][::intp], label = "vr2")
ax1.grid()
ax2.grid()
ax1.legend()
ax2.legend()
fig.suptitle("Reward Signals")
```

[39]:
```
Text(0.5, 0.98, 'Reward Signals')
```



[38]:
```
fig, (ax1, ax2) = plt.subplots(2, figsize=(12,10))

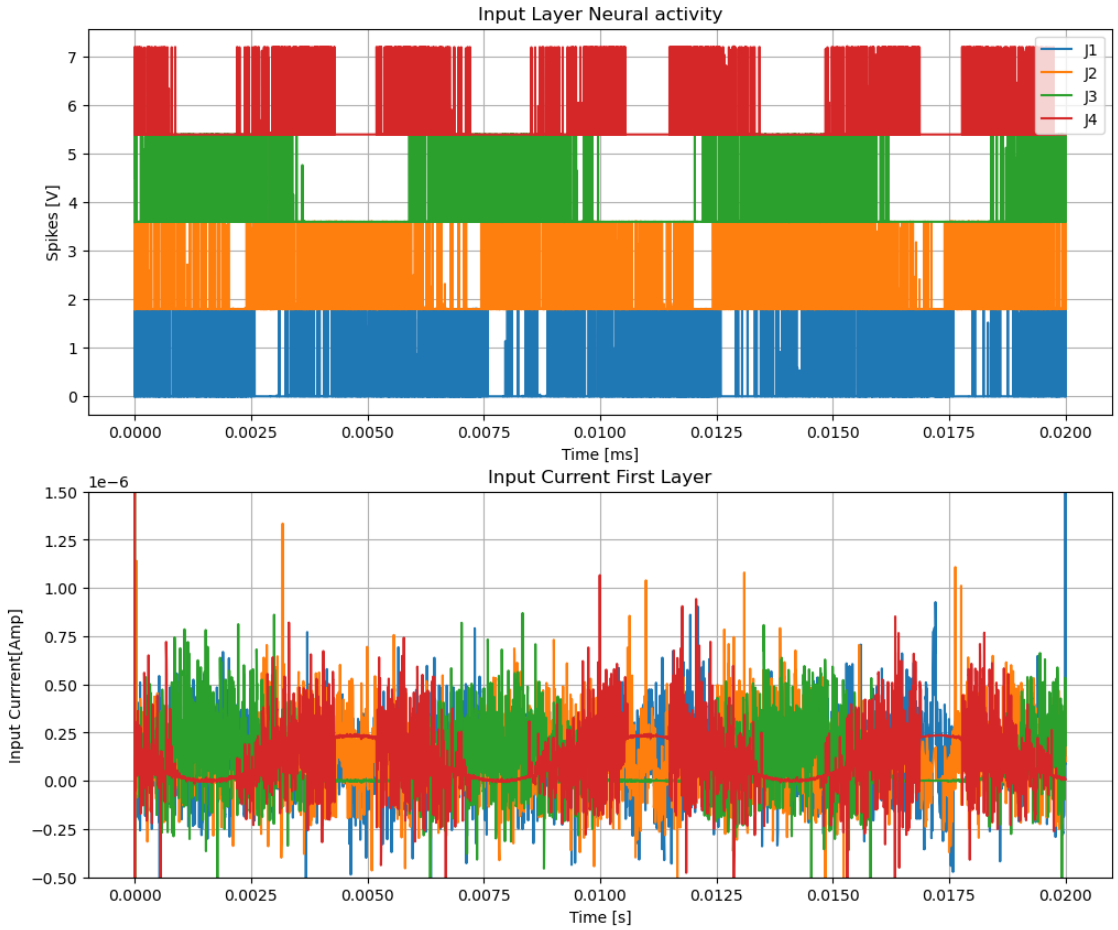
ax1.plot(data["time"][::intp], data["v(j1)"][::intp]+(1.8*0), label = f"J{1}")
ax1.plot(data["time"][::intp], data["v(j2)"][::intp]+(1.8*1), label = f"J{2}")
ax1.plot(data["time"][::intp], data["v(j3)"][::intp]+(1.8*2), label = f"J{3}")
ax1.plot(data["time"][::intp], data["v(j4)"][::intp]+(1.8*3), label = f"J{4}")
ax1.legend()
ax1.grid()
ax1.set_xlabel("Time [ms]")
ax1.set_ylabel("Spikes [V]")
ax1.set_title("Input Layer Neural activity")


ax2.plot(data["time"][::intp], data["i(v.xx1.vext)"][::intp], label = f"i{1}")
ax2.plot(data["time"][::intp], data["i(v.xx2.vext)"][::intp], label = f"i{2}")
```

```
ax2.plot(data["time"][::intp], data["i(v.xx3.vext)"][::intp], label = f"i{2}")
ax2.plot(data["time"][::intp], data["i(v.xx4.vext)"][::intp], label = f"i{2}")
ax2.grid()
ax2.set_xlabel("Time [s]")
ax2.set_ylabel("Input Currrent[Amp]")
ax2.set_title("Input Current First Layer")
ax2.set_ylim((-0.05e-5, 0.15e-5))

# fig.savefig('InputLayer.pdf')
```

[38]: (-5e-07, 1.5e-06)



[49]:
```
fig, (ax1, ax2) = plt.subplots(2, figsize=(12,10))

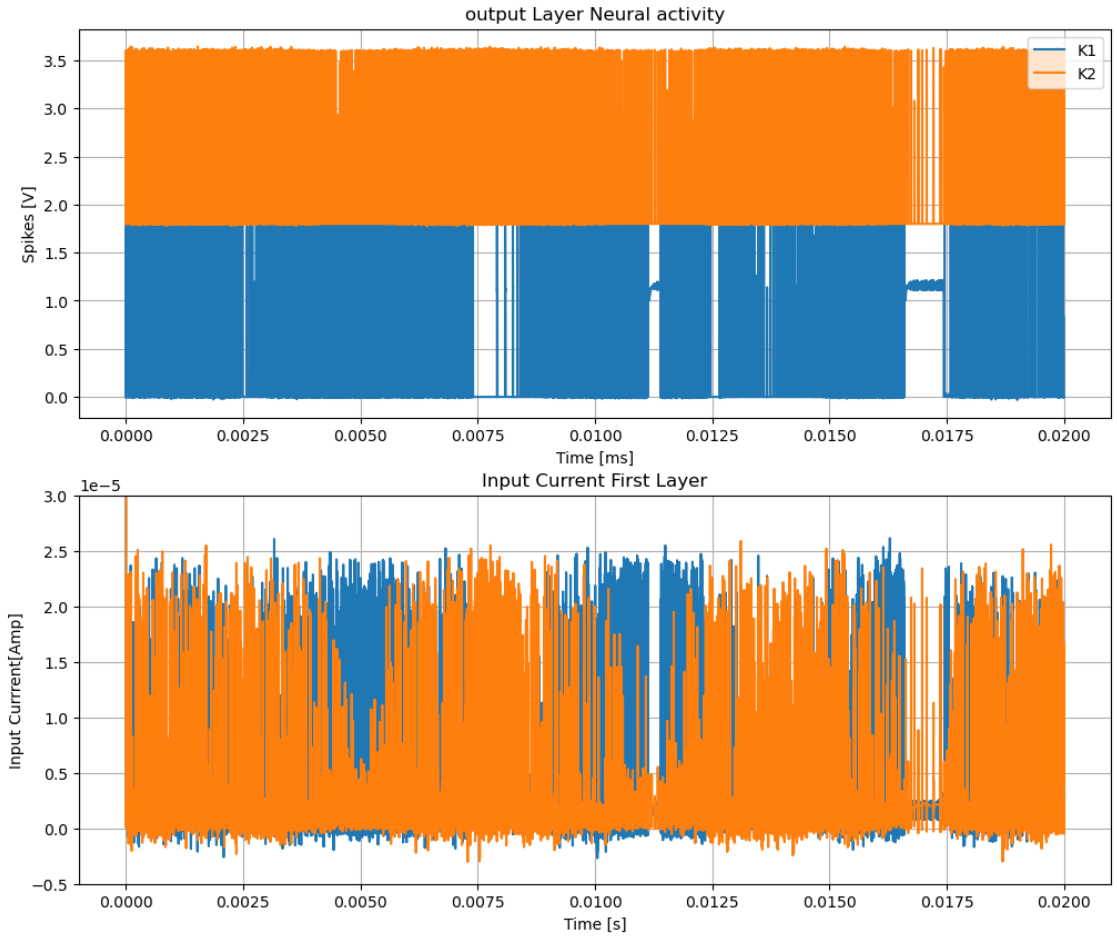
ax1.plot(data["time"][::intp], data["v(k1)"][::intp], label = "K1")
ax1.plot(data["time"][::intp], data["v(k2)"][::intp]+1.8, label = "K2")
ax1.legend()
ax1.grid()
```

```
ax1.set_xlabel("Time [ms]")
ax1.set_ylabel("Spikes [V]")
ax1.set_title("output Layer Neural activity")


ax2.plot(data["time"][::intp], data["i(v.xx5.vext)"][::intp], label = f"i{1}")
ax2.plot(data["time"][::intp], data["i(v.xx6.vext)"][::intp], label = f"i{2}")
ax2.grid()
ax2.set_xlabel("Time [s]")
ax2.set_ylabel("Input Currrent[Amp]")
ax2.set_title("Input Current First Layer")
ax2.set_ylim((-0.5e-5, 3e-5))

# fig.savefig('InputLayer.pdf')
```

[49]: (-5e-06, 3e-05)



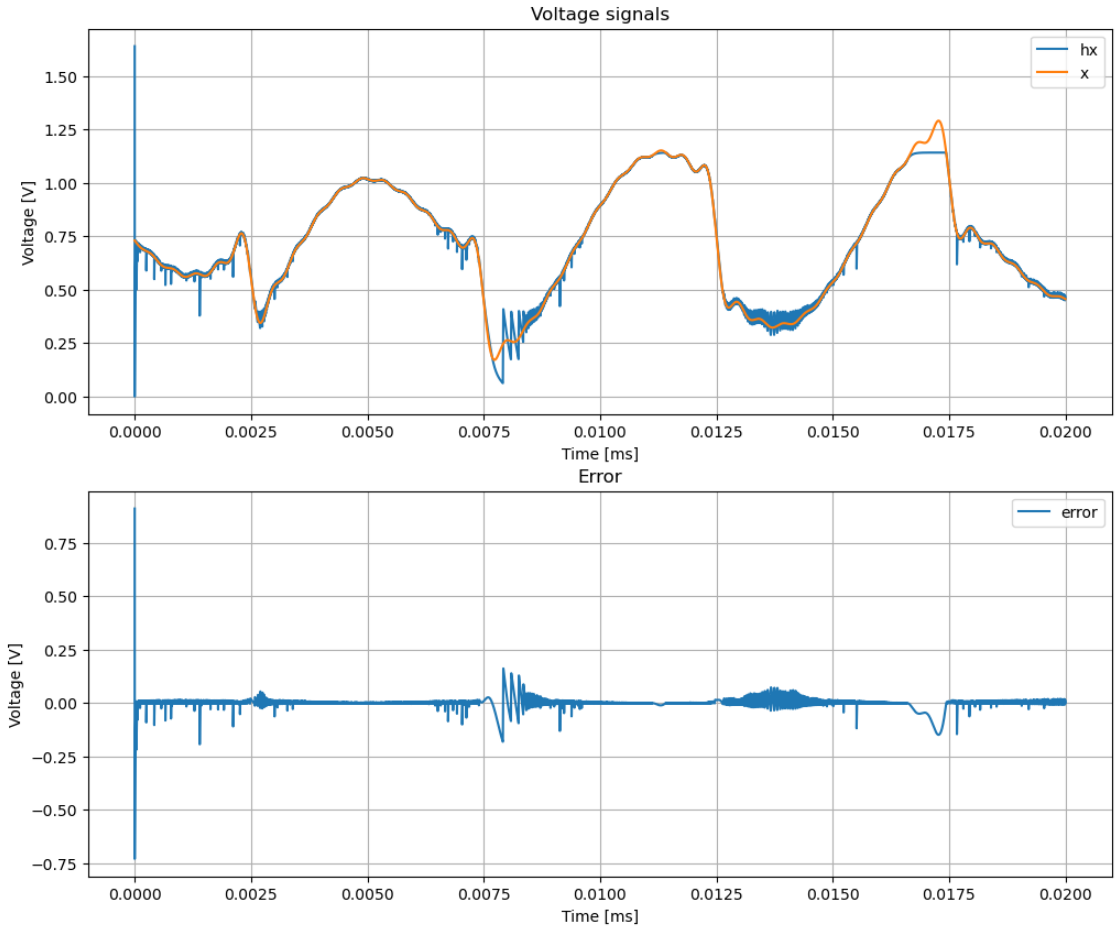
[34]:
```
fig, [ax1, ax2] = plt.subplots(2, figsize=(12,10))

ax1.plot(data["time"][::intp], data["v(hx1)"][::intp], label = "hx")
ax1.plot(data["time"][::intp], data["v(x1)"][::intp], label = "x")
# ax1.plot(data["time"][::intp], data["v(vin1)"][::intp], label = "vin")
ax1.legend()
ax1.grid()
ax1.set_xlabel("Time [ms]")
ax1.set_ylabel("Voltage [V]")
ax1.set_title("Voltage signals")


ax2.plot(data["time"][::intp], data["v(hx1)"][::intp] - data["v(x1)"][::intp],␣
 ↪label = "error")
ax2.legend()
ax2.grid()
ax2.set_xlabel("Time [ms]")
ax2.set_ylabel("Voltage [V]")
ax2.set_title("Error")


# fig.savefig('Signals.pdf')
```

[34]:
```
Text(0.5, 1.0, 'Error')
```



[31]:
```
fig, [ax1, ax2] = plt.subplots(2, figsize=(12,10))

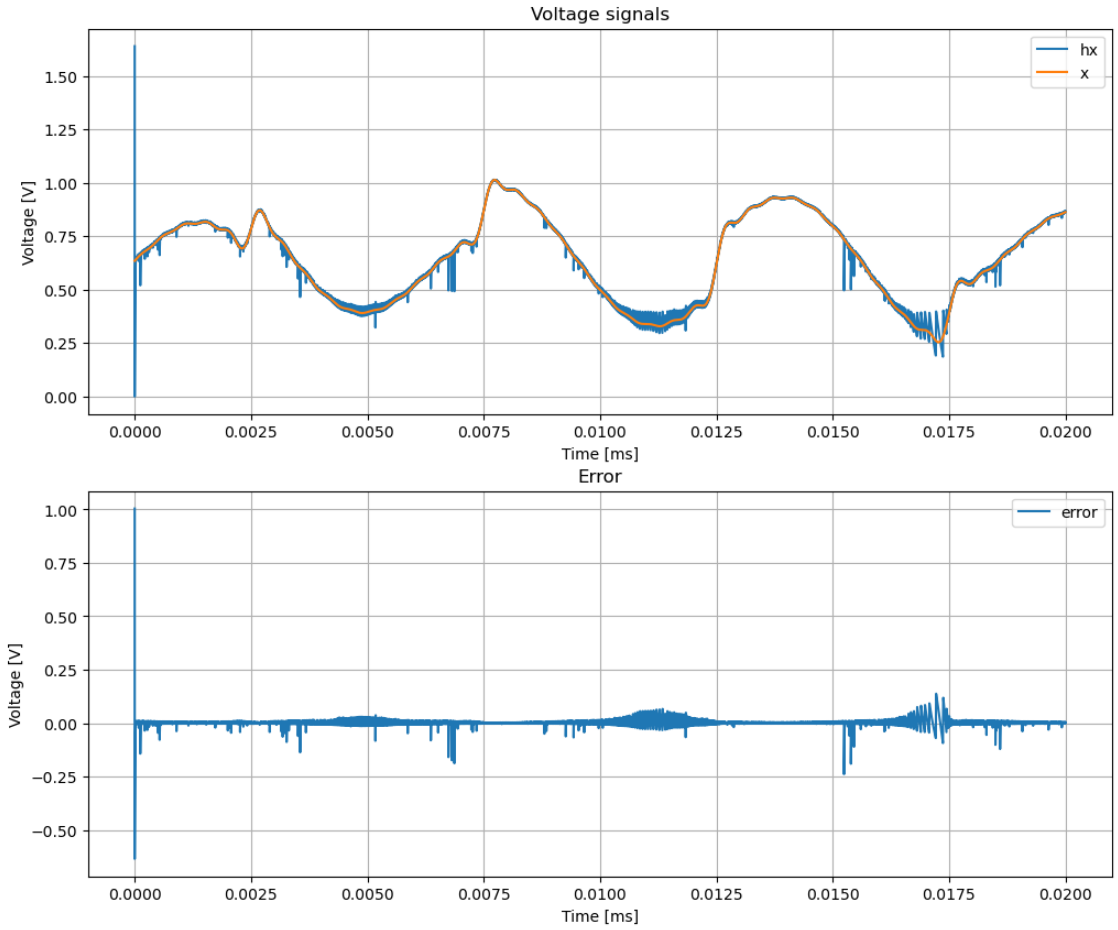
ax1.plot(data["time"][::intp], data["v(hx2)"][::intp], label = "hx")
ax1.plot(data["time"][::intp], data["v(x2)"][::intp], label = "x")
# ax1.plot(data["time"][::intp], data["v(vin1)"][::intp], label = "vin")
ax1.legend()
ax1.grid()
ax1.set_xlabel("Time [ms]")
ax1.set_ylabel("Voltage [V]")
ax1.set_title("Voltage signals")


ax2.plot(data["time"][::intp], data["v(hx2)"][::intp] - data["v(x2)"][::intp],␣
 ↪label = "error")
ax2.legend()
ax2.grid()
ax2.set_xlabel("Time [ms]")
ax2.set_ylabel("Voltage [V]")
```

```
ax2.set_title("Error")


# fig.savefig('Signals.pdf')
```

[31]:
```
Text(0.5, 1.0, 'Error')
```



[32]:
```
fig, ax2 = plt.subplots(1, figsize=(12,4))

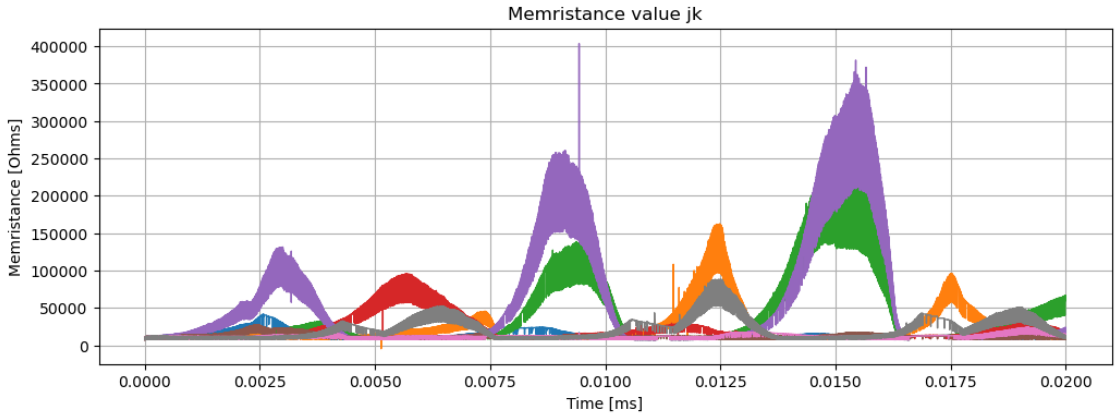
mem_jk = []
for k in range(8):
    Imem_label = "i(v.x11.xstdp" +  str(k+1) + ".vmr)"
    Vte_label =  "v(x11.xstdp" +  str(k+1) + ".te)"
    Vbe_label = "v(x11.xstdp" +  str(k+1) + ".be)"
    Imem= data[Imem_label][::intp]
    Vte = data[Vte_label][::intp]
    Vbe = data[Vbe_label][::intp]
    mem = (Vbe - Vte)/Imem
```

```
    ax2.plot(data["time"][::intp], mem, label = f"jk{k+1}",linewidth=1,␣
 ↪markersize=2)
    mem_jk.append(mem)


# ax.legend()
ax2.grid()
ax2.set_xlabel("Time [ms]")
ax2.set_ylabel("Memristance [Ohms]")
ax2.set_title("Memristance value jk")
# ax2.set_ylim((-0.2e6, 3.5e6))
# fig.savefig('weights.pdf')
```

```
/tmp/ipykernel_89648/4151008435.py:11: RuntimeWarning: divide by zero
encountered in divide
  mem = (Vbe - Vte)/Imem
```

[32]:
```
Text(0.5, 1.0, 'Memristance value jk')
```



[ ]: